

Software quality assurance is:

A systematic, planned set of actions necessary to provide adequate confidence that the software development process or the maintenance process of a software system product conforms to established functional technical requirements as well as with the managerial requirements of keeping the schedule and operating within the budgetary confines.

Gaining SQA from theory to implementation

© Pearson Education Limited 2004

১. মূল চিন্তা বাংলায় — সহজ করে

এখানে মূল কথা হলো: ১২ জন মানুষকে এমনভাবে ভাগ করতে হবে, যাতে ৯ মাসের মধ্যে JUSIS তৈরি করা যায় এবং Quality ও Security ঠিক থাকে।

১. Management & Requirement — ২ জন

- **Project Manager (PM) – ১ জন:** পুরো project পরিচালনা করবে—সময়, budget, risk, milestone ইত্যাদি দেখবে।
- **Business Analyst (BA) – ১ জন:** VC, শিক্ষক, ছাত্র, ব্যাংক ইত্যাদি stakeholder-এর কাছ থেকে requirement সংগ্রহ করবে এবং developer-দের বুঝিয়ে দেবে।

কেন ২ জন?

Project manage করা এবং requirement gather করা—দুটি আলাদা গুরুত্বপূর্ণ কাজ।

🏢 ২. Architecture & Database — ২ জন

- **System Architect – ১ জন:** পুরো system-এর structure/design তৈরি করবে এবং payment gateway-এর সাথে system কীভাবে connect করবে তা ঠিক করবে।
- **DBA/Data Migration Expert – ১ জন:** পুরোনো ২০ বছরের student data নতুন database-এ নিরাপদে migrate করবে।

🔑 **বিশেষ কারণ:** ২০ বছরের data migration একটি **high-risk task**। Data হারালে বড় সমস্যা হবে।

💻 ৩. Development — ৫ জন

- **Backend Developer – ৩ জন:** API, business logic, payment gateway integration ইত্যাদি করবে।
- **Frontend Developer – ১ জন:** Student portal-এর UI তৈরি করবে।
- **Full-Stack Developer – ১ জন:** যেখানে workload বেশি হবে সেখানে frontend বা backend—দুই দিকেই support করবে।

🔑 **কেন Full-Stack Developer?**

এটা হলো **flexible resource**। কোনো team-এ কাজ বেশি হলে তাকে সেখানে দেওয়া যাবে। ফলে resource utilization ভালো হবে।

🛡️ ৪. Quality & Security — ৩ জন

- **SQA/Tester – ২ জন:** Developer-এর কাজ independently test করবে, review করবে, inspection/walkthrough করবে।
- **Security/DevOps Engineer – ১ জন:** Security, deployment, CI/CD এবং payment-related security দেখবে।

🔑 **সবচেয়ে গুরুত্বপূর্ণ SQA point:**

যে developer code লিখেছে, সে একাই নিজের code review করবে না। Independent tester থাকলে ভুল ও bias কমে।

💰 মোট হিসাব

Area	জনবল
Management & Analysis	2
Architecture & Database	2
Development	5
SQA & Security	3
Total	12

মনে রাখার shortcut:

$$2 + 2 + 5 + 3 = 12$$

Management → Architecture/Data → Development → Quality/Security

২. Exam-Ready English Answer

Question:

Jahangirnagar University is developing a web-based Student Information System (JUSIS) integrating a national mobile payment gateway. You have a cross-functional team of 12 members and a strict 9-month deadline. Based on the principles of the Step Wise framework (Step 7: Allocate Resources) and SQA fundamentals, how would you strategically structure this 12-member team? Provide specific roles and justify your allocation.

Exam-Ready Answer

To successfully complete the JUSIS project within the strict 9-month deadline, the 12-member cross-functional team should be allocated according to their responsibilities, project risks, and quality requirements.

1. Project Management and Analysis – 2 Members

- **1 Project Manager (PM):** Responsible for project planning, schedule, budget, risk management, milestones, and overall project coordination.
- **1 Business Analyst (BA):** Collects and analyzes requirements from stakeholders such as students, teachers, university management, and banks, and communicates them clearly to the development team.

2. Architecture and Database Management – 2 Members

- **1 System Architect:** Designs the overall system architecture and ensures proper integration with external services such as the mobile payment gateway.
- **1 Database Administrator (DBA) / Data Migration Expert:** Manages the database and safely migrates 20 years of legacy student data into the new system without data loss.

3. Core Development Team – 5 Members

- **3 Backend Developers:** Develop APIs, business logic, database-related services, and integrate the system with banks and mobile payment gateways.
- **1 Frontend Developer:** Develops the user-friendly and responsive Student Portal interface.
- **1 Full-Stack Developer:** Acts as a flexible resource and supports either frontend or backend development according to workload and project priorities.

4. Quality Assurance and Security – 3 Members

- **2 SQA Engineers/Testers:** Independently perform testing, peer reviews, inspections, walkthroughs, and prepare test plans to identify defects early.
- **1 Security/DevOps Engineer:** Ensures data security, manages deployment and CI/CD pipelines, and protects sensitive financial and student information.

Justification

This allocation follows the **Step Wise principle of resource allocation** by assigning the right people to the right tasks according to project needs and risks.

It also follows **SQA principles**, especially the separation of development and quality assurance. Developers should not be the sole reviewers of their own work because independent reviewers can identify defects more objectively.

The dedicated **DBA** reduces the risk of losing important 20-year legacy data, while the **Security/DevOps Engineer** reduces security risks associated with payment transactions and sensitive student information. Finally, the **Full-Stack Developer** provides flexibility and improves resource utilization when workload changes during the project.

Therefore, the proposed allocation is:

2 Management + 2 Architecture/Data + 5 Development + 3 SQA/Security = 12 Members.

This structure provides a balanced combination of **management, technical expertise, development capacity, quality assurance, security, and resource flexibility**, increasing the likelihood of completing JUSIS successfully within the 9-month deadline.

📁 Category 1: Software Uniqueness & Quality

Q1. Why is software engineering fundamentally more problematic than traditional industrial engineering?

◆ প্রশ্নটা কী জানতে চাচ্ছে?

প্রশ্নটি জানতে চাচ্ছে—সফটওয়্যার তৈরি করা কেন একটি physical machine/product তৈরি করার চেয়ে বেশি কঠিন?

□ সহজ লজিক

একটা গাড়ি বা মেশিনে কোনো পার্ট missing থাকলে চোখে দেখা যায়। কিন্তু সফটওয়্যারে কোনো hidden bug বা execution path অনেকদিন লুকিয়ে থাকতে পারে।

🔪 Exam-Ready Answer

- **High Complexity:** Software has almost infinite data paths and execution loops.
- **Invisibility:** Missing parts or “phantom paths” in code are not visually obvious and may remain hidden for years.
- **Limited Defect Correction:** Unlike industrial products, software manufacturing mainly copies the master code. Therefore, the development phase is the main opportunity to detect and fix defects.

★ মনে রাখবে

Software = Complex + Invisible + Hidden Defects

Q2. Explain the impact of “Interfaces with other systems” and “Team Member Changes” in an SQA environment.

◆ প্রশ্নটা কী জানতে চাচ্ছে?

দুইটা জিনিসের impact জানতে চাচ্ছে:

1. অন্য system-এর সাথে software-এর **interface/integration** হলে কী সমস্যা হতে পারে?
2. Project চলাকালীন **team member পরিবর্তন** হলে কী সমস্যা হয়?

□ সহজ লজিক

Attendance System → HRMS → Payroll

একটার output যদি অন্যটার input হয়, তাহলে interface-এ সমস্যা হলে পুরো chain-এ সমস্যা হবে।

আর নতুন developer এলে তাকে system বুঝাতে সময় লাগে।

🔪 Exam-Ready Answer

- **Interfaces:** Outputs from one system often become inputs to another system. Therefore, poorly defined or non-standard interfaces can cause failures in interconnected systems.
- **Team Changes:** Staff changes can reduce productivity because new members need time to understand the project. According to Brooks' Law, **"Adding people to a late project makes it later."**

★ মনে রাখবে

Interface problem = System-to-system problem

Team change = Productivity problem

Q3. Differentiate between a Software Error, Fault, and Failure with a practical scenario.

◆ প্রশ্নটা কী জানতে চাচ্ছে?

Error, Fault এবং Failure-এর difference এবং একটি example দিতে হবে।

□ সহজ লজিক

ধরো OCR integration-এর সময়:

Programmer ভুল করল → Error

↓

ভুলটি code-এ থেকে গেল → Fault

↓

User চালানোর সময় system crash করল → Failure

🔪 Exam-Ready Answer

- **Software Error:** A mistake made by a programmer, such as using an incorrect operator or logic.
- **Software Fault:** The error that exists in the software code and may remain inactive.

- **Software Failure:** An incorrect system behavior that occurs when a fault is activated during execution.

★ Shortcut

Error → Fault → Failure

ভুল করা → Code-এ ভুল থাকা → Run করার সময় সমস্যা

Q4. Discuss “Faulty requirements” and “Client-developer communication failures” as root causes of software errors.

◆ প্রশ্নটা কী জানতে চাচ্ছে?

Software-এর error কেন শুরুতেই তৈরি হয়—তার দুটি root cause explain করতে হবে।

□ সহজ লজিক

Client বলল: “একটা simple report চাই।”

Developer বুঝল একরকম।

Client চেয়েছিল আরেকরকম।

অথবা client নিজেই incomplete requirement দিল।

ফলে শুরুতেই ভুল software তৈরি হবে।

✍ Exam-Ready Answer

- **Faulty Requirements:** Requirements may be incomplete, incorrect, missing, or unnecessary. Such faulty requirements can increase project complexity, cost, and errors.
- **Communication Failures:** Misunderstanding between clients and developers can lead to incorrect interpretation of requirements and mismatched expectations.

★ মনে রাখবে

Wrong Requirement + Wrong Communication = Software Error

Q5. What is the “Expanded Definition of SQA”, and how does it differ from Quality Control (QC)?

◆ প্রশ্নটা কী জানতে চাচ্ছে?

এখানে SQA এবং QC-এর difference জানতে চাচ্ছে।

□ সহজ লজিক

QC: কাজ বানানোর পরে/চলাকালীন check করে defect বের করা।

SQA: শুধু defect খোঁজা নয়; পুরো project যেন **quality + budget + schedule** মেনে চলে, সেটাও নিশ্চিত করা।

🔪 Exam-Ready Answer

- **Quality Control (QC):** Inspections and checks performed during development to evaluate the quality of a specific work product.
- **Expanded SQA:** A systematic and planned set of activities that ensures the project satisfies both **technical requirements** and **managerial requirements**, including budget and schedule.

★ Shortcut

QC = Check the product

SQA = Ensure overall quality

📁 Category 2: Software Quality Factors

Q6. Outline McCall’s three categories of Software Quality Factors.

◆ প্রশ্নটা কী জানতে চাচ্ছে?

McCall software quality factors-কে কোন ৩টি category-তে ভাগ করেছেন?

□ সহজ লজিক

Software-এর quality তিন দিক থেকে দেখা হয়:

1. এখন ঠিকমতো চলছে?

2. ভবিষ্যতে পরিবর্তন করা সহজ?
3. অন্য system-এ নেওয়া/যোগ করা সহজ?

🔪 Exam-Ready Answer

McCall's quality factors are divided into three categories:

1. **Product Operation:** How well the software operates.
Examples: Correctness, Reliability, Efficiency, Integrity, Usability.
2. **Product Revision:** How easily the software can be changed and tested.
Examples: Maintainability, Flexibility, Testability.
3. **Product Transition:** How easily the software can be moved or integrated.
Examples: Portability, Reusability, Interoperability.

★ Shortcut

Operation → Revision → Transition

Q7. Differentiate between Reliability and Efficiency within McCall's framework.

◆ প্রশ্নটা কী জানতে চাচ্ছে?

দুটি quality factor-এর difference:

Reliability vs Efficiency

□ সহজ লজিক

Reliability: System কতটা dependable? বারবার crash করে কি না?

Efficiency: কাজ করতে কত resource লাগে? CPU, memory, storage ইত্যাদি।

🔪 Exam-Ready Answer

- **Reliability:** Measures the ability of software to provide its required service without failure.
- **Efficiency:** Measures the amount of hardware resources required to perform the software functions.

★ Shortcut

Reliability = Failure কম
Efficiency = Resource কম

Q8. What are the alternative quality factors introduced by Evans & Marciniak and Deutsch & Willis?

◆ প্রশ্নটা কী জানতে চাচ্ছে?

দুইজন author/model কী কী **additional quality factors** introduce করেছেন, সেটা লিখতে হবে।

🔗 Exam-Ready Answer

- **Evans & Marciniak:** Introduced **Verifiability** and **Expandability**.
- **Deutsch & Willis:** Introduced **Safety**, **Manageability**, and **Survivability**.

□ সহজ অর্থ

- **Verifiability** = software verify করা সহজ।
 - **Expandability** = future-এ system বড় করা সহজ।
 - **Safety** = system যেন ক্ষতি না করে।
 - **Manageability** = system/manage/version control করা সহজ।
 - **Survivability** = সমস্যা হলেও service চালু রাখার ক্ষমতা।
-

📁 Category 3: Project Evaluation & Programme Management

Q9. Distinguish between a Programme Manager and a Project Manager.

◆ প্রশ্নটা কী জানতে চাচ্ছে?

Programme Manager বনাম Project Manager—কে কতগুলো project দেখে এবং resource কীভাবে manage করে?

□ সহজ লজিক

Project Manager = One project

Programme Manager = Multiple projects

Programme Manager পুরো group-এর resource optimize করে।

🔪 Exam-Ready Answer

- **Programme Manager:** Handles multiple simultaneous projects, optimizes resources across them, and maintains personal relationships with skilled staff.
- **Project Manager:** Handles one project at a time, focuses on minimizing resource demand for that project, and views the project as unique.

★ Shortcut

Programme = Many

Project = One

Q10. Explain Cost-Benefit Analysis (CBA), NPV, and IRR in project evaluation.

◆ প্রশ্নটা কী জানতে চাচ্ছে?

Project financially ভালো কি না বোঝার **তিনটি method** explain করতে হবে।

□ সহজ লজিক

CBA: খরচের চেয়ে লাভ বেশি কি?

NPV: Future-এর টাকা আজকের হিসেবে কত valuable?

IRR: কোন discount rate-এ NPV = 0?

🔪 Exam-Ready Answer

- **CBA:** Compares the total expected costs with the total expected benefits of a project.
- **NPV:** Calculates the present value of future cash flows using a discount factor:
$$\frac{1}{(1+r)^t}$$
- **IRR:** The discount rate at which the project's NPV becomes exactly zero. If the IRR is higher than the organization's borrowing rate, the project is generally considered viable.

★ Shortcut

CBA = Cost vs Benefit
NPV = Present Value
IRR = NPV = 0

Q11. How do Decision Trees and Project Risk Matrices evaluate project uncertainties?

◆ প্রশ্নটা কী জানতে চাচ্ছে?

দুইটা tool-এর কাজ জানতে চাচ্ছে:

Risk Matrix → Risk কতটা dangerous?

Decision Tree → কোন decision নেওয়া ভালো?

🔪 Exam-Ready Answer

- **Risk Matrix:** Evaluates risks based on their **Likelihood/Probability** and **Impact/Importance**.
- **Decision Tree:** Graphically represents different choices and their possible financial outcomes and probabilities. It can be used to calculate **Expected Monetary Value (EMV)**.

□ Example

Build vs Buy

- Build করলে কত cost?
- Buy করলে কত cost?
- কোনটার success probability বেশি?

Decision Tree সাহায্য করবে।

📁 Category 4: Step Wise Planning & Estimation

Q12. How does Stakeholder Analysis influence project objectives in the Step Wise framework?

◆ প্রশ্নটা কী জানতে চাচ্ছে?

Stakeholder-দের analyse করার পরে **project objective** পরিবর্তন হতে পারে কেন?

□ সহজ লজিক

ধরো, software-এর objective ছিল শুধু “online examination system তৈরি করা।”

কিন্তু stakeholder analysis করে জানা গেল students privacy নিয়ে concerned।

তাহলে objective-এর মধ্যে **secure/private result handling** যোগ করতে হবে।

🔪 Exam-Ready Answer

- Stakeholder analysis reveals important interests, concerns, and resistance.
- These findings may reveal requirements that were not considered initially.
- Therefore, project objectives may need to be revised or expanded to gain stakeholder support.

★ Shortcut

Stakeholder Analysis → New Concern → Objective Revision

Q13. Differentiate between Bottom-up and Top-down estimating. When is each appropriate?

◆ প্রশ্নটা কী জানতে চাচ্ছে?

দুটি estimation method-এর **difference** + **কখন ব্যবহার করব** জানতে চাচ্ছে।

□ সহজ লজিক

Bottom-up

Project → ছোট task → আরও ছোট task → সব যোগ

Top-down

Whole Project = 100 days

→ Design 30

→ Coding 30

→ Testing 40

🔪 Exam-Ready Answer

- **Bottom-up Estimating:** The project is divided into small activities, usually tasks that one person can complete in one or two weeks. The estimates are then added together. It is appropriate when historical data is unavailable.
- **Top-down Estimating:** An overall estimate is produced using cost drivers or historical data and then distributed among project components. It is appropriate when reliable previous project data is available.

★ Shortcut

Bottom-up = Task → Total

Top-down = Total → Task

Q14. Explain Parkinson's Law and Weinberg's Zeroth Law in software estimation.

◆ প্রশ্নটা কী জানতে চাচ্ছে?

দুটি famous law কী বলে এবং estimation-এর সাথে তাদের relation কী?

🔪 Exam-Ready Answer

- **Parkinson's Law:** "Work expands to fill the time available." Therefore, an excessive estimate may cause the team to use more time than necessary.
- **Weinberg's Zeroth Law:** "A software project that does not have to meet a reliability requirement can meet any other requirement." It highlights the importance of reliability requirements in determining realistic software effort.

□ সহজে

Parkinson: বেশি সময় দিলে → বেশি সময় লাগে।

Weinberg: Reliability requirement বাদ দিলে → অন্য requirement fulfil করা সহজ দেখাবে।

Q15. Compare Function Points (Mark II & COSMIC) with COCOMO.

◆ প্রশ্নটা কী জানতে চাচ্ছে?

তিনটা estimation approach-এর মূল focus কী, সেটা compare করতে হবে।

□ সহজ লজিক

Function Point → Software-এর Size

COSMIC → Data Movement

COCOMO → Effort

🔊 Exam-Ready Answer

- **Function Points Mark II:** Measures software size using data inputs, outputs, and entity types accessed.
- **COSMIC Function Points:** Measures functional size in layered or embedded software by counting **Entries, Exits, Reads, and Writes**.
- **COCOMO:** Estimates software effort using system size such as KLOC and productivity factors. Its basic form is:
$$\text{Effort} = c \times \text{Size}^k$$

★ Shortcut

Mark II → Inputs/Outputs/Entities

COSMIC → E-E-R-W

COCOMO → KLOC + Effort

📖 Category 5: Planning, Risks & Reviews

Q16. What are the essential elements of a comprehensive Software Development Plan?

◆ প্রশ্নটা কী জানতে চাচ্ছে?

একটি complete Software Development Plan-এ কী কী বিষয় থাকতে হবে?

□ সহজ লজিক

কি বানাবো → কার সাথে connect করব → কীভাবে বানাবো → কারা বানাবে → কী risk আছে

🔗 Exam-Ready Answer

A comprehensive Software Development Plan should include:

1. **Project Products:** Code, manuals, training materials, and other deliverables.
2. **Project Interfaces:** Integration with existing databases, systems, or APIs.
3. **Methodology & Tools:** Development methodology and technology/tools.
4. **Staff Organization:** Team structure, roles, skills, and resource allocation.
5. **Risk Management:** Identification and mitigation of development risks.

★ Memory Trick

Products → Interfaces → Method → Staff → Risk

Q17. Explain “Gold Plating” and list three other top software risks according to Boehm and Ross.

◆ প্রশ্নটা কী জানতে চাচ্ছে?

Gold Plating কী এবং আরও ৩টি major software risk লিখতে হবে।

□ সহজ লজিক

Client বলেনি, তবুও developer নিজের ইচ্ছায় feature add করল।

এটাই **Gold Plating**।

🔗 Exam-Ready Answer

- **Gold Plating:** Adding unnecessary or unrequested features to a software product, which can increase project cost, complexity, and schedule.

Three other major risks are:

1. **Unrealistic schedules and budgets**
2. **Developing the wrong software functions**
3. **Continuous changes in requirements**

★ Shortcut

Gold Plating = Unwanted Features

Q18. What are the major differences between Formal Design Reviews (FDRs) and Peer Reviews?

◆ প্রশ্নটা কী জানতে চাচ্ছে?

FDR এবং Peer Review-এর participants ও authority compare করতে হবে।

✍ Exam-Ready Answer

FDR	Peer Review
Involves senior management and customer representatives	Involves peers/equal-level developers
Has authority to approve the document/design	Has no formal approval authority
Used for formal evaluation and approval	Mainly used for defect detection

□ Shortcut

FDR = Review + Approval

Peer Review = Review + Defect Detection

Q19. Contrast Inspections with Walkthroughs in Peer Reviews.

◆ প্রশ্নটা কী জানতে চাচ্ছে?

Peer Review-এর দুটি method:

Inspection বনাম Walkthrough

□ সহজ লজিক

Inspection = Formal + Strict

Walkthrough = Informal + Flexible

✍ Exam-Ready Answer

- **Inspection:** It is a highly formal review process with strict procedures and checklists. The author does not act as the presenter, and follow-up is required to ensure corrections.
- **Walkthrough:** It is a less formal review process where the author normally presents and explains the work. It generally has less strict follow-up.

★ Shortcut

Inspection → Formal, Checklist, Follow-up

Walkthrough → Informal, Author Presents

Q20. According to Pressman, what are the golden guidelines for a successful Design Review Session?

◆ প্রশ্নটা কী জানতে চাচ্ছে?

Design Review meeting কীভাবে properly conduct করতে হবে—Pressman-এর **guidelines** লিখতে হবে।

□ সহজ লজিক

Review meeting-এর উদ্দেশ্য হলো **ভুল খুঁজে বের করা**, কে ভুল করেছে সেটা নিয়ে ঝগড়া করা বা solution নিয়ে debate করা নয়।

🔪 Exam-Ready Answer

According to Pressman, the major guidelines are:

1. **Focus on defect detection** rather than solving the defects during the session.
2. **Avoid discussing possible solutions** during the review.
3. **Discuss issues professionally** without personal attacks.
4. If disagreement occurs, **record the issue and discuss it in another forum**.
5. The review session should **not exceed two hours**.

★ Super Shortcut

Find Defects → No Solution Debate → No Personal Attack → Record Disagreement → Max 2 Hours

শেষ মুহূর্তের Mega Revision

পরীক্ষার আগে শুধু এই keywords মনে রাখলে অনেক প্রশ্ন cover করতে পারবে:

Topic	এক লাইনে মনে রাখবে
Software Problem	Complex + Invisible + Hidden defects
Error/Fault/Failure	Mistake → Code defect → Execution problem
SQA	Technical + Managerial quality
McCall	Operation + Revision + Transition
Reliability	Failure-free service
Efficiency	Resource usage
Programme Manager	Many projects + Resource optimization
Project Manager	One project
CBA	Cost vs Benefit
NPV	Present value of future cash flow
IRR	Rate where NPV = 0
Risk Matrix	Probability × Impact
Decision Tree	Choices + Probability + EMV
Stakeholder Analysis	Concerns → Objective revision
Bottom-up	Small tasks → Total
Top-down	Total → Components
Parkinson	Work expands to fill available time
Mark II FP	Inputs + Outputs + Entities
COSMIC	Entries + Exits + Reads + Writes
COCOMO	Effort = $c \times \text{Size}^k$
Gold Plating	Unwanted features

Topic	এক লাইনে মনে রাখবে
FDR	Review + Approval
Peer Review	Defect detection
Inspection	Formal
Walkthrough	Informal
Design Review	Defect detection, max 2 hours